

排序算法备课讲义

您好！为了“备课”这个目标，我们不仅要看懂代码，更要能用清晰的语言把算法的“故事”讲出来。我将以您的讲义为蓝本，对每个算法的核心思想、执行步骤和关键细节进行拆解，希望帮助您更生动地讲解这些内容。

1. 插入排序 (Insertion Sort)

教学核心

紧扣“整理扑克牌”这个比喻：始终维护一个 已排序区间 ，将新元素“插入”到恰当位置。

步骤详解

分区（逻辑上）

- 已排序区间（左）：初始仅 $a[0]$ 。
- 未排序区间（右）： $a[1] \dots a[n-1]$ 。

摸牌（外层 for ）

- 循环 $n-1$ 次：每次取“未排序区间”的第一个元素作为 $temp$ ，常见写法 $temp = a[i+1]$ 。

插牌（内层 while ）

- 令 $end = i$ ，自右向左比较：
- 当 $while (end \geq 0 \ \&\& \ a[end] > temp)$ ：
 - 将较大的 $a[end]$ 右移： $a[end + 1] = a[end]$ ；
 - 指针左移： $end--$ 。
- 本质：把已排序区间中所有大于 $temp$ 的元素依次右移一格。

落位

- 循环结束后， $temp$ 应插入到 $end + 1$ 处。

教学要点（Why）

为何稳定

- 条件为 $a[end] > temp$ ，遇到相等时停止， $temp$ 插到相等元素之后（ $end + 1$ ），相对次序不变。

为何最好是 $O(n)$

- 当序列“基本有序”， $while$ 条件一次也不满足，内层退化为 $O(1)$ ，总复杂度 $O(n)$ 。

2. 希尔排序 (Shell Sort)

教学核心

“插入排序的加强版”。通过分组与大步长的插入式预排序，让元素跳跃式接近目标位置，最后用 $gap=1$ 的插入排序快速收尾。

步骤详解

分组（Gap 概念）

- 采用增量序列，讲义中为 $gap = n/2$ 逐次折半。
- 例如 $n=6$ 、 $gap=3$ 时，索引按间隔 3 划为 3 组；
- 代码用 `for (int i = gap; i < n; i++)` 将各组的插入过程交织地完成。

预排序（大 gap 的“插入”）

- 外层缩 gap ：`for (int gap = n/2; gap > 0; gap /= 2) ;`
- 遍历未排序端：`for (int i = gap; i < n; i++) ;`
- 组内比较移动与插入排序相同，但步长为 gap ：
 - 移动：`a[end + gap] = a[end] ;`
 - 退位：`end -= gap ;`
- 对应插入排序的 `a[end+1] = a[end]` 与 `end--` 的“步长版”。

收尾（ $gap = 1$ ）

- 最终必到 $gap = 1$ ；
- 此时内层循环与标准插入排序等价；由于预排序令序列已“基本有序”，因此整体极快，命中插入排序的最好情形。

💡 教学要点（Why）

为何不稳定

- 大步长移动会让元素跨越与其相等的元素（这些相等元素通常位于不同组）。例如反例：`[5_A, 7, 6, 2, 5_B, 4]`，首轮 $gap=3$ 后得到 `[2, 5_B, 4, 5_A, 7, 6]`，`5_B` 跳到 `5_A` 之前，稳定性被破坏。
- **记忆法**：当执行 `a[end + gap] = a[end]` 与 `end -= gap` 时，元素会“跨过” $gap - 1$ 个中间位置，可能改变相等元素的相对次序。

3. 堆排序 (Heap Sort)

🎯 教学核心

“选择型排序”：用“堆”高效地反复选出未排序区间的最大值。

两大阶段：**建堆** → **排序**

阶段一：建堆（构造大顶堆）

- **大顶堆定义**：完全二叉树且每个父节点 $\geq$ 子节点。
- **自底向上建堆**：
 - 最后一个非叶子节点索引为 $(size - 2) / 2$ ；
 - 从该处向前遍历至 0，对每个 i 执行 `AdjustDown(a, size, i)` 。

- **AdjustDown （向下调整）：**
 - 左子： `child = 2 * parent + 1` ；
 - 取较大子：若 `a[child] < a[child + 1]` 则 `child++` ；
 - 若 `a[child] > a[parent]` 则交换，并令 `parent = child` 、 `child = parent * 2 + 1` 继续下沉；
 - 否则 `break` 结束。
- **结果：** `a[0]` 成为全局最大。

阶段二：排序

- 设 `end` 为未排序区间的末尾，循环 `while (end > 0)` ：
- **交换堆顶与区间末尾：** `Swap(&a[0], &a[end])` ，最大元素就位；
- 将堆大小缩为 `end` ，并对 0 执行 `AdjustDown(a, end, 0)` ；
- `end--` ，重复至完成。

💡 教学要点（Why）

为何不稳定

- 交换堆顶与末尾会让相等元素跨越，从而改变相对顺序。

空间复杂度 $O(1)$

- 原地重排，无需额外数组。
- **教学说明：**如实现为迭代版 `AdjustDown` 则严格 $O(1)$ ；若写成递归，会有 $O(\log n)$ 调用栈，但辅助结构空间仍为 $O(1)$ 。

4. 选择排序 (Selection Sort)

🎯 教学核心

朴素直观的“每轮把最小的放到前面”。您的讲义代码是“双向选择”的优化版：一趟同时找最小值与最大值，分别放到区间两端。

步骤详解（以优化版为例）

划定区间

- `begin = 0` , `end = size - 1` ，未排序区间为 `[begin, end]` 。

一趟搜寻

- `while (begin < end)` ，设 `mini = begin` , `maxi = begin` ，遍历 `i = begin ... end` ，用
 - `if (a[i] < a[mini]) mini = i;`
 - `if (a[i] > a[maxi]) maxi = i;`
- 记录当轮最小与最大索引。

放置最小值

- `Swap(a + begin, a + mini)` ， `a[begin]` 就位。

放置最大值（陷阱修正）

- 若 `begin == maxi` ，说明原最大值被刚才的交换挪到了 `mini` ，需修正 `maxi = mini` ；
- 再执行 `Swap(a + end, a + maxi)` 。

收缩区间

- `begin++`; `end--`; 。

💡 教学要点（Why）

为何不稳定

- （以单向版为例更直观）：如 `[5_A, 5_B, 2]` ，第一趟把 `2` 换到前面后得到 `[2, 5_B, 5_A]` ，相等元素次序改变。

5. 快速排序 (Quick Sort)

🎯 教学核心

分治法的经典实现：

- **Divide**：选取基准（Pivot），划分为 `[≤ Pivot]` 与 `[≥ Pivot]` ；
- **Conquer**：分别递归左右区间；
- **Combine**：原地分区，左右排好即整体有序。

讲义使用“挖坑法”（`PartSort1`）。

步骤详解（挖坑法 `PartSort1`）

选基准，挖第一个坑

- `int pivot = begin`; （此处 `begin` 即 `left`）；
- `int key = a[pivot]`; ；置 `a[pivot]` 为“坑”。

交替填坑（`while (begin < end)`）

- **右向左找**：移动 `end` ，定位到 `< key` 的元素；
- **右填左坑**： `a[pivot] = a[end]` ，并将 `pivot = end` ；
- **左向右找**：移动 `begin` ，定位到 `> key` 的元素；
- **左填右坑**： `a[pivot] = a[begin]` ，并将 `pivot = begin` ；
- 如此往复，直至相遇。

填上最后的坑

- 相遇处即为基准最终位置，将 `key` 放入： `a[pivot] = key` ；
- 返回 `pivot` 位置。

递归（`QuickSort`）

- **出口**： `if (left >= right) return`; ；
- **分区**： `int keyindex = PartSort1(...)` ；
- **递归左右**： `QuickSort(a, left, keyindex - 1)`; 与 `QuickSort(a, keyindex + 1, right)`; 。

💡 教学要点 (Why)

最坏 $O(n^2)$

- 若每次基准都是区间极值（如已升序数组且总取 `a[left]` ），分割极不平衡，递归深度为 n ，总和为 $n + (n-1) + \dots + 1$ 。
- （讲解小贴士）可提示常见优化：随机化基准或三数取中以降低极端分割概率。

6. 归并排序 (Merge Sort)

🎯 教学核心

亦为分治，但先分后合：先递归到底，再进行有序子列的合并。

步骤详解

分（_MergeSort 递归）

- **出口：** `if (left >= right) return; ;`
- **取中点：** `int mid = (left + right) >> 1; ;`
- **递归左右：** `_MergeSort(a, left, mid, temp);` 与 `_MergeSort(a, mid + 1, right, temp);`。

合（双指针归并）

- 两段均已各自有序： `[left ... mid]` 与 `[mid+1 ... right]` ；
- **指针：** `begin1 = left , begin2 = mid + 1 , index = left ;`
- **主循环：** `while (begin1 <= end1 && begin2 <= end2) ,`
 - 若 `a[begin1] <= a[begin2]` 则 `temp[index++] = a[begin1++]`;
 - 否则取右段；
- **收尾：** 将剩余元素直接拷入 `temp` ；
- **拷回原数组：** `for (int i = left; i <= right; i++) a[i] = temp[i]; ;`
- `temp` 为全局复用的一块缓冲区。

💡 教学要点 (Why)

为何稳定

- 关键在比较时使用 `<=` ，当两侧元素相等时，始终先拷左边到 `temp` ，保持相对顺序。

为何空间 $O(n)$

- 需要大小为 n 的临时数组 `temp` ，并在所有合并步骤中复用。